3. Estructuras condicionales

En programación, una **estructura condicional** permite ejecutar diferentes bloques de código según si una condición se cumple o no. Esto es fundamental para la toma de decisiones en un programa.

Python usa la palabra clave **if** para definir una condición, y opcionalmente se pueden agregar **else** y **elif** para manejar distintos casos. Por medio de ellas es posible que el programa defina qué bloques de código ejecutar primero o directamente no hacerlo. Para ello es necesario también entender sobre la indentación, los operadores relacionales y los operadores lógicos.

3.1. Indentación

Python **no usa llaves {} ni palabras clave como begin y end** para delimitar bloques de código, a diferencia de lenguajes como C, Java o Pascal. En su lugar, **usa la indentación**, es decir, la sangría o el desplazamiento hacia la derecha en cada línea de código que pertenece a un bloque.
Cada vez que una línea de código termina con `:` (dos puntos), Python espera que el siguiente bloque de código esté **indentado**.

```
edad = 20

if edad>=18:
    print("Eres mayor de edad")
    print("Puedes votar")

print("Fin del programa")
```

3.1.1. Algunas reglas de Indentación

- **La indentación es obligatoria:** Si no indentas correctamente, Python mostrará un error de sangría (**IndentationError**).
- **Todos los elementos de un bloque deben tener la misma indentación:** Si mezclamos espacios y tabulaciones o usamos niveles distintos en un mismo bloque, Python lo detectará como un error.
- **Se recomienda usar 4 espacios por nivel de indentación:** La convención en Python (según PEP 8) es usar **4 espacios** en cada nivel.

3.2. Estructuras

3.2.1. if

Tal como vimos en los ejemplos anteriores, la estructura if evalúa una condición y, si es True, ejecuta el bloque de código correspondiente.

3.2.2. if con else, dos caminos posibles

A veces, necesitamos que una acción se ejecute si la condición es **False**. Para esto, usamos **else**.

```
edad = int(input("Ingrese su edad: "))

if edad>=18:
    print("Eres mayor de edad")
else:
    print("Eres menor de edad")
```

En este caso, se cumpla o no la condición, el programa notifica al usuario, ya sea por el lado verdadero como por el lado falso.

```
r Semestre - 26/Programacion
Ingrese su edad: 15
Eres menor de edad
```

3.2.3. if, elif y else, múltiples condiciones y caminos posibles

Cuando hay **más de dos posibles escenarios**, usamos **elif** (abreviatura de *else if*).

```
nota = 85

if nota >= 90:
    print("Calificación: A")
elif nota >= 80:
    print("Calificación: B")
elif nota >= 70:
    print("Calificación: C")
else:
    print("Calificación: Reprobado")
```

Importante:

- **Solo una condición se ejecuta**, la primera que sea **True**.
- Si ninguna se cumple, Python ejecuta el bloque **else**.
- **Podemos combinar condiciones** con operadores lógicos.
- Si es necesario se pueden anidar condiciones cuando son expresiones muy complejas y sea de difícil lectura en una sola condición.

3.2.4. match

A partir de Python 3.10, podemos usar **match-case**, que se asemeja a **switch** de otros lenguajes. Esta opción puede ser más acorde cuando hay múltiples caminos logrando mayor legibilidad.

```
print("Opciones")
print("1. opción 1")
print("2. opción 2")
print("3. opción 3")
print("4. opción 4")

opcion = int(input("Elija una opción"))

match opcion:
    case 1:
        print("Elegiste la opción 1")
    case 2:
        print("Elegiste la opción 2")
    case 3:
        print("Elegiste la opción 3")
    case 4:
        print("Elegiste la opción 4")
    case _:
        print("Opción no válida")
```

Explicación:

- **match opción:** evalúa el valor de opción.
- Cada **case** equivale a un **case en switch** de otros lenguajes.
- _ (guión bajo) actúa como **default**, cubriendo todas las opciones no listadas.

3.3. Expresiones

Una expresión en Python puede ser tan simple como un valor literal o una variable, o tan compleja como una combinación de múltiples operadores y funciones.

3.3.1. Ejemplos de expresiones simples

```
x = 10
nombre = "Leopoldo"
es_estudiante = True
```

3.3.2. Ejemplos de expresiones compuestas

3.4. Precedencia y Asociatividad de Operadores

```
x = 10 + 5 * 2
resultado = (x > 10) and (x < 30)
mensaje = f"El resultado es {x}"
```

Orden

Precedencia	Operadores	Descripción
1	()	Paréntesis, se evalúan primero
2	**	Potencias
3	*, /, //, %	Multiplicación, divisiones y módulo
4	+, -	Sumas y restas
5	==, !=, >, >=, <, <=	Operadores relacionales
6	not	Negación lógica
7	and	Operador lógico AND
8	or	Operador lógico OR
9	=, +=, -=, *=, /=, //=, %=, **=	Operadores de asignación

```
x = 10 + 5 * 2 #El resultado es 20
x = (10 + 5) * 2 #El resultado es 30
x = 10 + 5 ** 2 #El resultado es 35
x = 8
x += x * 6 #El resultado es 56
```

Si dos operadores tienen la misma precedencia, se evalúan según su **asociatividad**:

- **Asociatividad izquierda**: Se evalúa de izquierda a derecha (la mayoría de los operadores).
- **Asociatividad derecha**: Se evalúa de derecha a izquierda (como ** y =).

3.5. Evaluación en Expresiones Booleanas

Las expresiones booleanas siguen una evaluación perezosa (**short-circuit evaluation**). Esto significa que Python **deja de evaluar** una expresión lógica tan pronto como el resultado es determinante.

```python
print("Números pares O mayores de 16")
numero = int(input("Ingrese un numero: "))

if numero > 16 or numero % 2 == 0:
    print(f"Ingresó: {numero}... El número entra")
else:
    print(f"Ingresó: {numero}... El número no entra")
print("--------------------------------")
print("Números pares Y mayores de 16")
numero = int(input("Ingrese un numero: "))

if numero > 16 and numero % 2 == 0:
    print(f"Ingresó: {numero}... El número entra")
else:
    print(f"Ingresó: {numero}... El número no entra")
```

En el primer ejemplo, si el número es mayor a 16, cumpliéndose la primera condición: numero > 16 ya no evaluará la segunda que es. numero % 2 == 0 porque una de las dos se cumple y así actúa OR para que sea True.
Por el contrario, en el segundo ejemplo, ambas condiciones deben cumplirse para ser True.

3.6. Tabla de verdad

A	B	A and B	A or B	Not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

- **and (Y lógico):** Sólo es True si **ambas condiciones** son verdaderas.
- **or (O lógico):** Es True si **al menos una** de las condiciones es verdadera.
- **not (Negación lógica):** Invierte el valor lógico: **True** → **False** y **False** → **True**.